**PYTHON PROGRAMMING**

**CASE STUDY ASSIGNMENT – 6**

Case Study 6 – Fuel Consumption and Route Comparison

## 1. Problem Statement

Fuel consumption and travel distance are important factors when planning a journey. Different routes may have different distances, which can affect the amount of fuel required and the total travel cost. Calculating these values manually can be difficult when comparing multiple routes.

The Fuel Consumption and Route Comparison system is a Python-based program that accepts the distance and fuel efficiency for different routes. It calculates the estimated fuel required and fuel cost for each route, then identifies the route with the lowest fuel cost. This helps users compare routes and make fuel-efficient travel decisions.

## 2. Objectives

1. To calculate the fuel required for travelling along different routes.
2. To calculate the estimated fuel cost for each route.
3. To compare routes based on fuel consumption and cost.
4. To identify the route with the lowest estimated fuel cost.
5. To validate user inputs such as distance, mileage, and fuel price.
6. To apply Python concepts such as loops, conditional statements, lists, and arithmetic operations in a practical application.

### 3. Assumptions

1. The user must enter at least one route for comparison.
2. The distance of each route is entered in kilometres.

3. The vehicle's fuel efficiency is entered in kilometres per litre (km/l).
4. The fuel price is entered in rupees per litre (₹/litre).
5. Route distance, fuel efficiency, and fuel price must be positive numbers.
6. Fuel required is calculated by dividing the route distance by fuel efficiency.
7. Estimated fuel cost is calculated by multiplying the fuel required by the fuel price.
8. The route with the lowest estimated fuel cost is considered the most fuel-efficient option based on cost.
9. Traffic, road conditions, toll charges, and other additional expenses are not considered.

**4. Algorithm**

1. Start the program.
2. Read the number of routes to compare.
3. If the number of routes is less than or equal to zero, display an error message.
4. Otherwise, input the fuel price per litre.
5. For each route:
    - Enter the route name.
    - Enter the distance in kilometres.
    - Enter the vehicle's mileage in kilometres per litre.
    - Validate the entered values.
    - Calculate fuel required using:

Fuel Required = Distance / Mileage

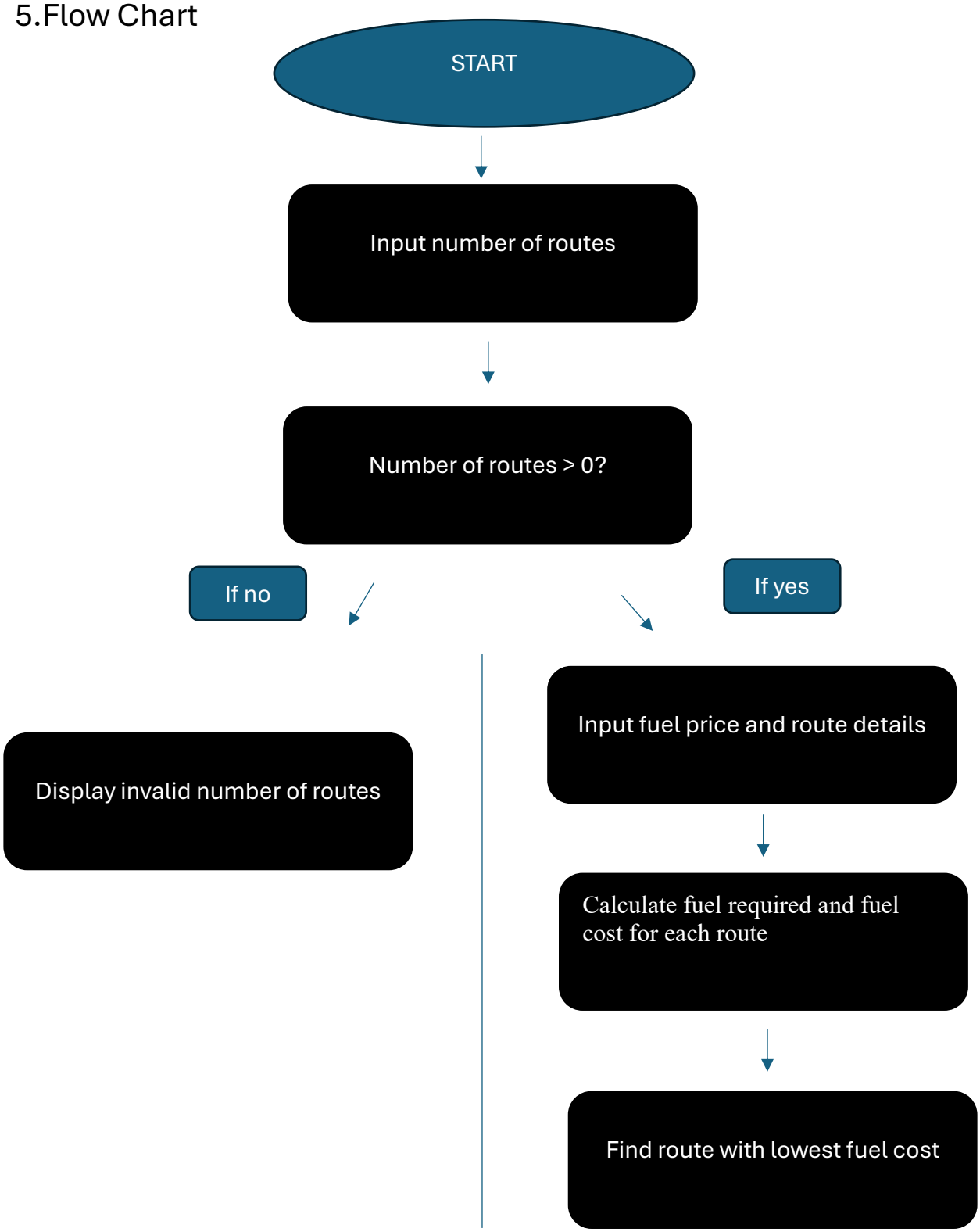
    - Calculate fuel cost using:

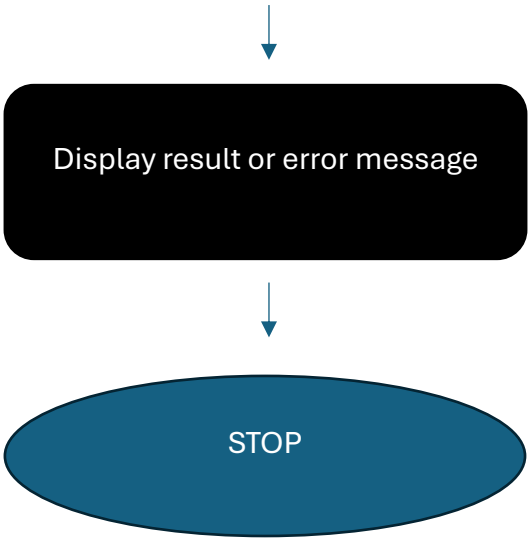
Fuel Cost = Fuel Required × Fuel Price

- Display the fuel required and estimated fuel cost.

6. Compare the fuel costs of all valid routes.
7. Identify and display the route with the lowest fuel cost.
8. Stop the program.

## 5.Flow Chart






6. Pseudo code

```
BEGIN
INPUT number_ of_ routes

IF number _of_ routes <= 0 THEN
   PRINT "Invalid number of routes"

ELSE
   INPUT fuel_ price

   IF fuel _price <= 0 THEN
      PRINT "Invalid fuel price"
ELSE
      SET lowest _cost = infinity
```

```
    SET best _route = ""

    FOR each route from 1 to number _of_ routes DO
INPUT route _name
      INPUT distance
      INPUT mileage

      IF distance <= 0 OR mileage <= 0 THEN
        PRINT "Invalid route details"

      ELSE
        Fuel _required = distance / mileage
        Fuel _cost = fuel _required * fuel _price

        PRINT fuel _required
        PRINT fuel _cost

        IF fuel _cost < lowest_cost THEN
          Lowest _cost = fuel_ cost
          Best _route = route _name
        END IF
      END IF
```

END FOR

IF best_ route is not empty THEN

PRINT best_ route

PRINT lowest_ cost

END IF

END IF

END IF

END

## 7.Source code

Fuel Consumption and Route Comparison.py

PYTHON-ACTIVITY-BASED-LEARNING > Fuel Consumption and Route Comparison.py > ...

```python
n = int(input("Enter number of routes: "))

if n <= 0:
    print("Invalid number of routes")

else:
    fuel_price = float(input("Enter fuel price per litre: "))

    if fuel_price <= 0:
        print("Invalid fuel price")

    else:
        lowest_cost = None
        best_route = ""

        for i in range(n):
            route_name = input("Enter route name: ")
            distance = float(input("Enter distance in km: "))
            mileage = float(input("Enter mileage in km/litre: "))

            if distance <= 0 or mileage <= 0:
                print("Invalid route details")

            else:
                fuel_required = distance / mileage
                fuel_cost = fuel_required * fuel_price

                print("Route:", route_name)
                print("Fuel required:", round(fuel_required, 2), "litres")
                print("Fuel cost: Rs.", round(fuel_cost, 2))

                if lowest_cost is None or fuel_cost < lowest_cost:
                    lowest_cost = fuel_cost
                    best_route = route_name

        if best_route != "":
            print("Lowest-cost route:", best_route)
            print("Lowest fuel cost: Rs.", round(lowest_cost, 2))
```

## 8. Test Cases

| Test Case | Input | Expected Output |
|---|---|---|
| 1. Compare two routes | Fuel price: ₹100/litre; Route A: 100 km, mileage 20 km/litre; Route B: 150 km, mileage 25 km/litre | Route A: 5 litres, ₹500; Route B: 6 litres, ₹600; Lowest-cost route: Route A |
| 2. Lowest cost on second route | Fuel price: ₹100/litre; Route A: 200 km, mileage 20 km/litre; Route B: 100 km, mileage 25 km/litre | Route A: 10 litres, ₹1000; Route B: 4 litres, ₹400; Lowest-cost |

## 9. Output Screenshots

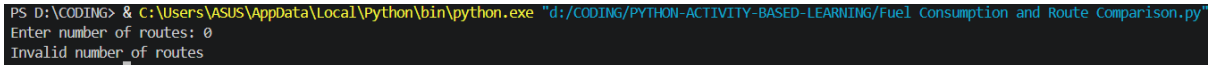
**Screenshot 1:** Comparison of two routes where Route A has the lowest fuel cost.



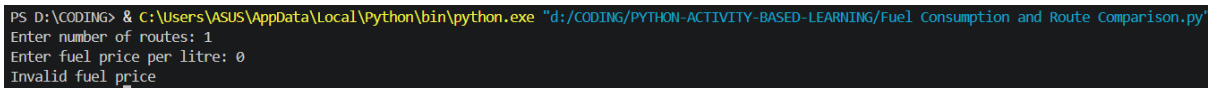

**Screenshot 2:** Comparison of two routes where Route B has the lowest fuel cost.



**Screenshot 3:** Program output for an invalid number of routes.




**Screenshot 4:** Program output for an invalid fuel price.




**Screenshot 5:** Program output for invalid route details.




## 10. Conclusion

The Fuel Consumption and Route Comparison program was developed successfully using Python. It calculates the fuel required and estimated fuel cost for different routes based on distance, vehicle mileage, and fuel price. The program compares the costs and identifies the route with the lowest fuel expense. It also validates user inputs to avoid invalid calculations. This project demonstrates the practical application of variables, loops, conditional statements, arithmetic operations, and input validation in Python. It helps users compare travel routes and make cost-effective decisions.